

Installation on Mac OS

Requirements

To Run the MyGPT pipeline, we will need the following minimum specifications for your system:

- 8 CPUs
- 8 GB Memory (16 GB or more for better response time)
- 10 GB hard-drive storage

We will also need several tools to run the pipeline:

- Brew
- Git
- Docker
- Ollama

Requirements installation

We will install these required tools in the following steps:

1. Brew installation

Check if you have `brew` installed on your system by running following command.

```
brew --version
```

If you get error that `brew not found`, you can install brew by following command from the new terminal window (similar to Jupyter installation but open new tab for terminal without closing existing window that's running Jupyter). For more details about brew check this [page](#)

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

2. Brew packages installation

We will install Brew packages by running following commands:

```
brew install git
```

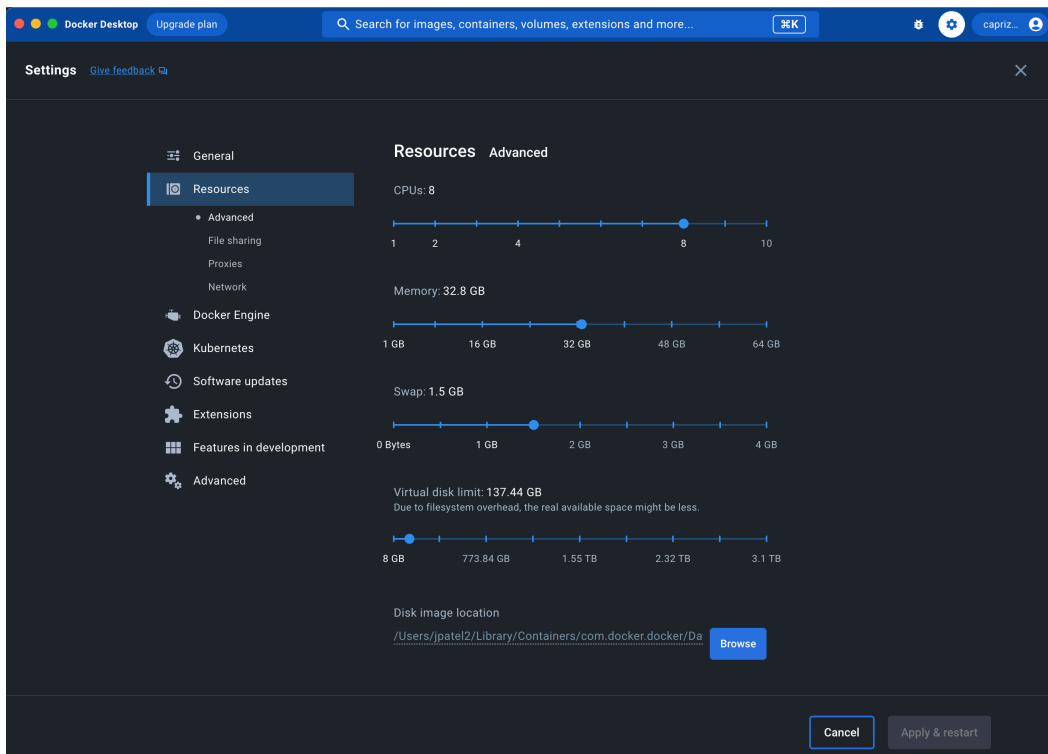
3. Docker installation

To run MyGPT with CPUs-only, the entire pipeline will run as a single unit from Docker. Check if you have `docker` installed on your system by running following command.

```
docker --version
```

If you get an error that `docker not found`, go to the official Docker installation page for Mac and install the appropriate Docker on your system: <https://docs.docker.com/desktop/install/mac-install/>

You can change Docker setting to match requirements for MyGPT:



4. Ollama installation

Finally, download and install Ollama by following instructions from this official [Ollama site](https://ollama.com)

You can check if Ollama is running by visiting <http://localhost:11434/> in your default browser.

[!CAUTION] After installing Ollama, close any open Terminal/Command Prompt before you pull Llama3.

Once you start Ollama, you have to pull Llama3 model by running following command:

```
ollama pull llama3
```

Also, get the nomic embedding model, which is best performing embedding model for MyGPT pipeline, by running following command:

```
ollama pull nomic-embed-text
```

MyGPT installation

You have 2 options to install MyGPT pipeline:

1. Use pre-built docker images (faster and easier to install, but you can't modify source code)
2. Build docker images from source code (slower, but you can modify source code)

Option 1: Use pre-built docker images

1. Get MyGPT source code

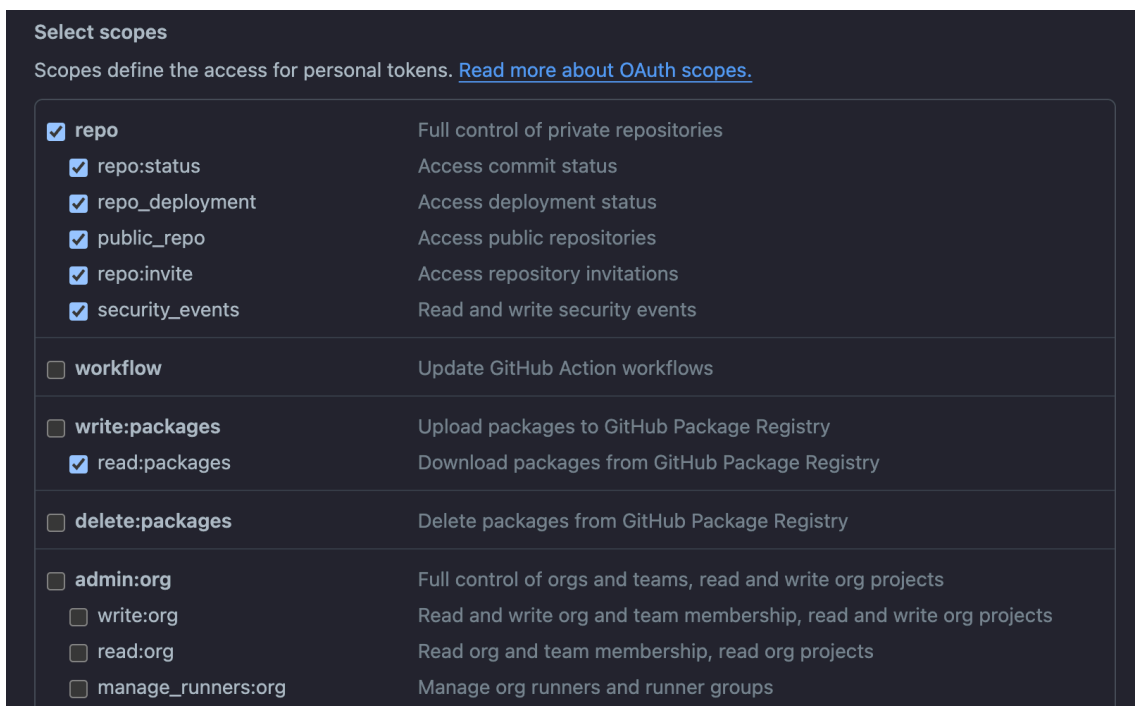
If you have zip file with MyGPT installation instructions, you can unzip it and copy it on your desktop or your desired location. You can skip remaining instructions from below and go to Step 2.

Or you can get installation instructions by running following command from Terminal. It will create `MyGPT` folder on your Desktop or desired folder. To get this from GitHub, we will need `GitHub username` and `GitHub access token` as the GitHub repo is private. When you run the following command, it will ask for this credentials.

```
git clone https://github.com/stjude-c3d/MyGPT.git
```

[!NOTE] If you don't have GitHub access token, you can generate classic token using this guideline: <https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens#creating-a-personal-access-token-classic>

[!CAUTION] While generating your GitHub Access Token, make sure you check access for `repo` and `read:packages` similar to image below.



The image shows the GitHub OAuth 'Select scopes' screen. It lists various permissions with checkboxes. The 'repo' scope is selected, along with its sub-scopes: 'repo:status', 'repo_deployment', 'public_repo', 'repo:invite', and 'security_events'. The 'workflow' scope is not selected. The 'write:packages' scope is not selected, but 'read:packages' is selected. The 'delete:packages' scope is not selected. The 'admin:org' scope is not selected, along with its sub-scopes: 'write:org', 'read:org', and 'manage_runners:org'.

Scope	Description
☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events
☐ workflow	Update GitHub Action workflows
☐ write:packages	Upload packages to GitHub Package Registry
☒ read:packages	Download packages from GitHub Package Registry
☐ delete:packages	Delete packages from GitHub Package Registry
☐ admin:org	Full control of orgs and teams, read and write org projects
☐ write:org	Read and write org and team membership, read and write org projects
☐ read:org	Read org and team membership, read org projects
☐ manage_runners:org	Manage org runners and runner groups

2. login to GitHub Docker registry

As the GitHub repository is private rightnow, we have to login to GitHub Docker registry to use the prebuilt images. To login to GitHub Docker registry, run following command. It will ask for your GitHub username and password.

[!CAUTION] Make sure Docker Desktop application is open and running before running the command. the password is your access token (same token you used in step 1), not your github password you use to login in github account.

```
docker login ghcr.io
```

3. Run docker containers

We will run following script to run docker containers:

```
cd MyGPT/installation/macOS/prebuilt_images
bash run_docker.sh
```

This script should take around 5-10 minutes to run. While above script is running, it will open several pages in your default browser. You can see status of different components of MyGPT pipeline on these pages.

- o backend: <http://localhost:8000/>

MyGPT Backend App

The App is running!

The app contains 5 collections.

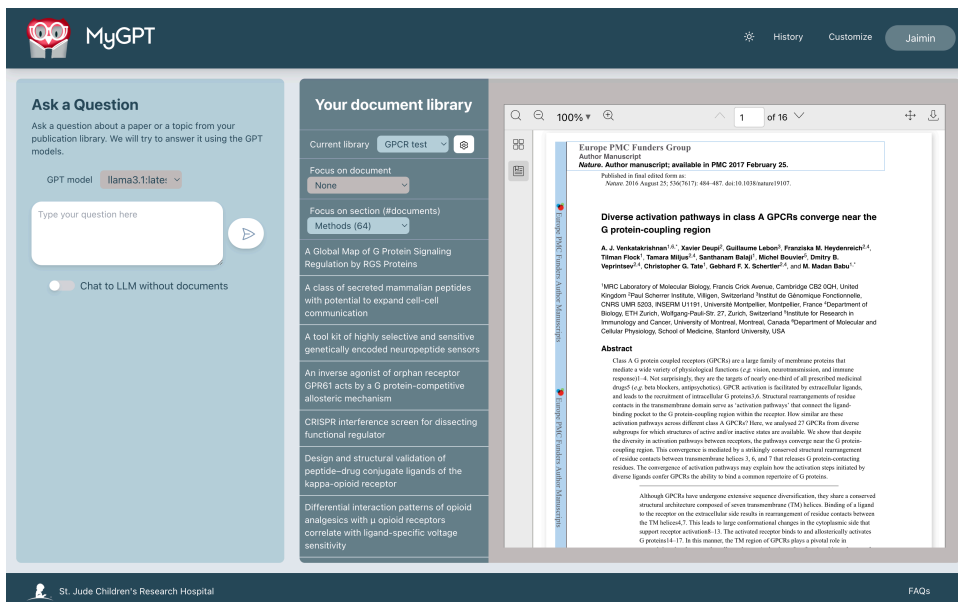
For Developers: API Endpoints

go to <http://localhost/api/> to access all available API endpoints.

Reload sample publication library

reload library

- o frontend: <http://localhost:3000/>



Option 2: Build docker images from source code

1. Get MyGPT source code

If you have zip file with MyGPT source code, you can unzip it and copy it on your desktop or your desired location. You can skip remaining instructions from below and go to Step 2.

If you want to get source code from GitHub we will first get MyGPT source code by running following command. It will create `MyGPT` folder on your Desktop. To get the source code from GitHub, we will need `GitHub username` and `GitHub access token` as the GitHub repo is private. When you run the following command, it will ask for this credentials.

```
git clone https://github.com/stjude-c3d/MyGPT.git
```

Note: If you don't have GitHub access token, you can generate classic token using this guideline: <https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens#creating-a-personal-access-token-classic>

2. Build docker images

We will run following script to build docker images:

```
cd MyGPT
bash installation/macOS/build_images/build_docker.sh
```

3. Run docker containers

We will run following script to run docker containers:

```
bash installation/macOS/build_images/run_docker.sh
```

This script should take around 5-10 minutes to run. While above script is running, it will open several pages in your default browser. You can see status of different components of MyGPT pipeline on these pages.

- backend: <http://localhost:8000/>

MyGPT Backend App

The App is running!

The app contains 5 collections.

For Developers: API Endpoints

go to <http://localhost/api/> to access all available API endpoints.

Reload sample publication library

reload library

- frontend: <http://localhost:3000/>

MyGPT

Ask a Question

Ask a question about a paper or a topic from your publication library. We will try to answer it using the GPT models.

GPT model: **llama3.1:latest**

Type your question here

☐ Chat to LLM without documents

Your document library

Current library: **GPCR test**

Focus on document: **None**

Focus on section (#documents): **Methods (64)**

A Global Map of G Protein Signaling Regulation by RGS Proteins

A class of secreted mammalian peptides with potential to expand cell-cell communication

A tool kit of highly selective and sensitive genetically encoded neuropeptide sensors

An inverse agonist of orphan receptor GPR61 acts by a G protein-competitive allosteric mechanism

CRISPR interference screen for dissecting functional regulator

Design and structural validation of peptide-drug conjugate ligands of the kappa-opioid receptor

Differential interaction patterns of opioid analgesics with μ opioid receptors correlate with ligand-specific voltage sensitivity

Europe PMC Funders Group
Author Manuscript
Nature. Author manuscript; available in PMC 2017 February 25.
Published on first listed item on
 Nature 2016 August 25; 536(7617): 484-487. doi:10.1038/nature19107.

Diverse activation pathways in class A GPCRs converge near the G protein-coupling region

A. J. Venkateshkrishnan^{1,2,3}, Xavier Druge¹, Guillaume Lebon¹, Franziska M. Heydenreich^{1,4}, Timon Fleck¹, Tamara Milgaj^{1,5}, Sebastian Baag¹, Michel Bouvier¹, Dmitry B. Veprikov^{1,6}, Christopher G. Tate¹, Gerhard F. X. Schertler^{2,6}, and M. Madan Babu^{1,7}

¹MRC Laboratory of Molecular Biology, Francis Crick Avenue, Cambridge CB2 0QH, United Kingdom ²Paul Scherrer Institute, Villigen, Switzerland ³Institut de Génétique Fonctionnelle, CNRS UMR 5205, INSERM U1191, Université Montpellier, Montpellier, France ⁴Department of Biology, ETH Zurich, Wolfgang-Paul-Str. 27, Zurich, Switzerland ⁵Institute for Research in Immunology and Cancer, University of Montreal, Montreal, Canada ⁶Department of Molecular and Cellular Physiology, School of Medicine, Stanford University, USA

Abstract

Class A G protein-coupled receptors (GPCRs) are a large family of membrane proteins that mediate a wide variety of physiological functions (e.g. vision, neurotransmission, and immune response). As yet, surprisingly, they are the targets of nearly one-third of all prescribed medicinal drugs (e.g. beta-blockers, antipsychotics). GPCR activation is facilitated by extracellular ligands, and leads to the recruitment of intracellular G proteins (G). Structural rearrangements of residues in the transmembrane domain serve as "activation pathways" that connect the ligand-binding pocket to the G protein-coupling region within the receptor. How similar are these activation pathways across different class A GPCRs? Here, we analysed 37 GPCRs from diverse subgroups for which structure of active and/or inactive states are available. We show that despite the diversity in activation pathways between receptors, the pathways converge near the G protein-coupling region. This convergence is mediated by a uniquely conserved structural rearrangement of residue contacts between transmembrane helices 5, 6, and 7 that release G protein-coupling residues. The convergence of activation pathways may explain how the activation steps initiated by diverse ligands confer GPCRs the ability to hold a common sequence of G proteins.

Although GPCRs have undergone extensive sequence diversification, they share a conserved structural architecture composed of seven transmembrane (TM) helices. Binding of a ligand to the receptor on the extracellular side results in rearrangement of residue contacts between the TM helices^{1,2}. This leads to large conformational changes in the cytoplasmic side that support receptor activation³⁻¹¹. The activated receptor binds to and allosterically activates G proteins¹²⁻¹⁷. In the process, the TM region of GPCR plays a pivotal role in

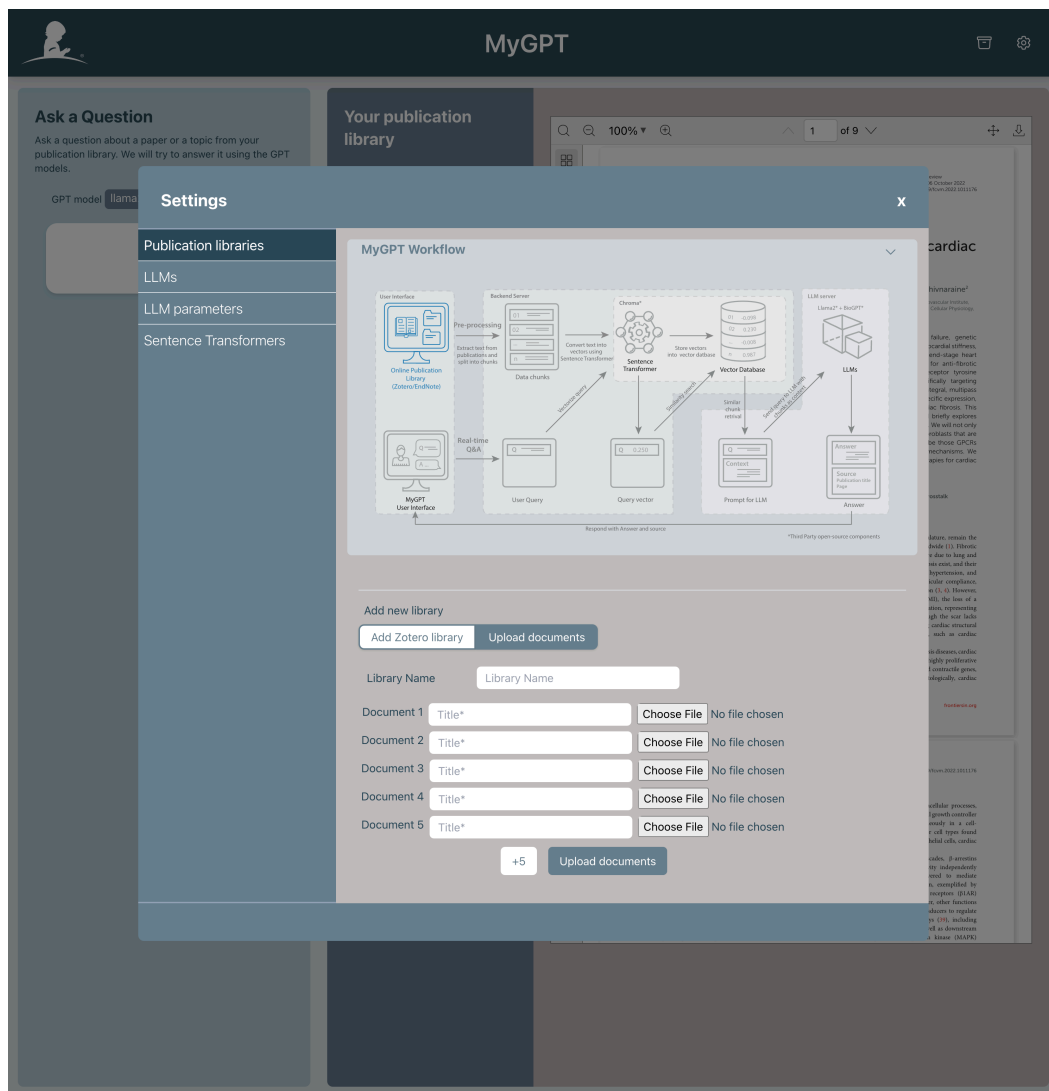
St. Jude Children's Research Hospital

FAQs

MyGPT pipeline usage

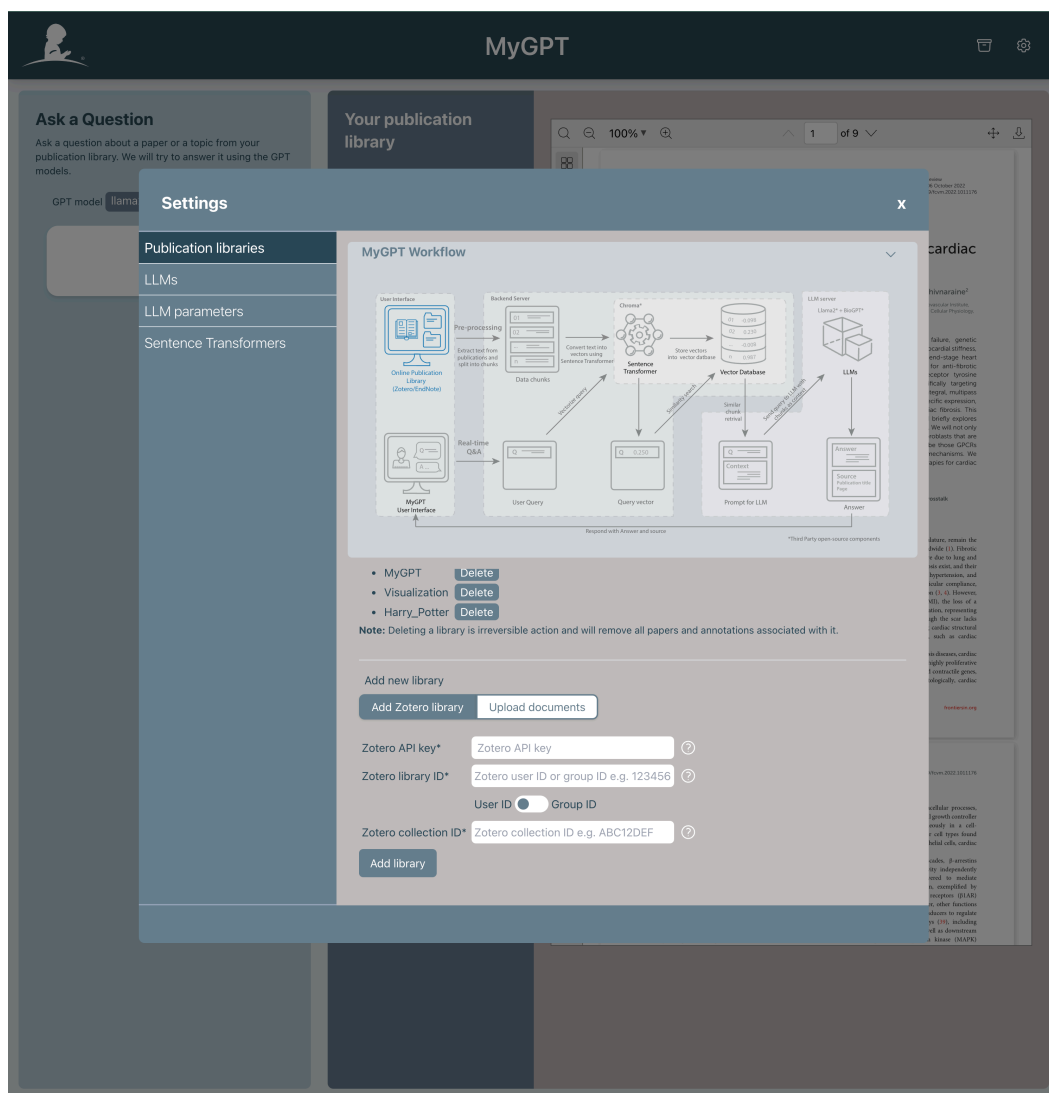
Upload publications

- You can upload documents or publications as PDF file by going to the settings, and selecting first tab: "Publication libraries" and selecting "Upload documents" menu as screenshot below.
- start by giving your library a "Library Name" and start uploading documents.
- You can upload upto 40 PDFs at a time. You can upload multiple times to same library. It will take some time to process the documents and once it's done, you can see your new library in the list with success message.



Add Zotero library

- You can add Zotero library by going to the settings, and selecting first tab: "Publication libraries" and selecting "Add Zotero library" menu as screenshot below.
- You must provide Zotero AI key, Zotero Group ID/User ID, and Zotero Collection ID to add Zotero library.
- If you need help getting any of these, you can click on the help button next to the input field or check section after screenshot.



- Zotero API key: You can generate an API key in your profile settings <https://www.zotero.org/settings/keys>
- Zotero User ID: You can get it from Zotero profile page by visiting <https://www.zotero.org/settings/keys>. It's 6-7 digit number.
- Zotero Group ID: You can get it from URL of the group, for example, here is URL for BABU group and group ID is 4982570 : https://www.zotero.org/groups/4982570/babu_group/
- Zotero Collection ID: You can get it from URL of the collection, for example, here is URL for BABU group and collection ID is YTPMLXYY : https://www.zotero.org/groups/4982570/babu_group/collections/YTPMLXYY

Other optional tasks for MyGPT pipeline

create super user

To create super user, run following command if you are using pre-built docker images:


```
cd MyGPT/installation/macOS/prebuilt_images/  
bash create_superuser.sh
```

Or run following command if you are building docker images:

```
bash MyGPT/installation/macOS/build_images/create_superuser.sh
```

You can check backend database at <http://localhost:8000/admin/> with username and password you created in above step.

Run MCP server

To run MCP server, run following command if you are using pre-built docker images:

```
cd MyGPT/installation/macOS/prebuilt_images/  
bash run_mcp_server.sh
```

Or run following command if you are building docker images:

```
bash MyGPT/installation/macOS/build_images/run_mcp_server.sh
```

This will start the MCP server and you can access it at <http://localhost:5001/sse/>

Configure MCP Client in MyGPT

To setup MCP client in MyGPT, you need to stop the frontend, and then update the environment variable in the `.env` file. To stop the frontend, run following command if you are using pre-built docker images:

```
bash stop_docker.sh
```

Or run following command if you are building docker images:

```
bash MyGPT/installation/macOS/build_images/stop_docker.sh
```

Change following line in `.env` file:

```
REACT_APP_MCP_SERVER_URL = 'http://localhost:5001/sse'  
REACT_APP_MCP_SHOW_MCP_MENU=true
```

You can change MCP server URL to your custom MCP server URL if you are using a different server.

Then, you can start the frontend again by running following command if you are using pre-built docker images:

```
bash run_docker.sh  
...
```

Or run following command if you are building docker images:

```
bash MyGPT/installation/macOS/build_images/run_docker.sh
```

Once the frontend is running, you can access it at <http://localhost:3000/> and you

should see MCP menu in the settings page.

Select the MCP tools you want to use from the customization page.

Once you select the tools, go to the main page and select LLM with tool support and you should see the MCP tools in the chat interface.

stop docker containers

To stop docker containers, run following command if you are using pre-built docker images:

```
cd MyGPT/installation/macOS/prebuilt_images/ bash stop_docker.sh
```

Or run following command if you are building docker images:

```
bash MyGPT/installation/macOS/build_images/stop_docker.sh ``
```